Desenvolvimento de Aplicação Web para indexação e busca em diários oficiais dos municípios

Leonardo Marques Ferreira



Universidade Federal de Uberlândia
Faculdade de Computação
Bacharelado em Sistemas de Informação

Uberlândia - MG
2025

Leonardo Marques Ferreira

Desenvolvimento de Aplicação Web para indexação e busca em diários oficiais dos municípios

Trabalho de Conclusão de Curso de Desenvolvimento de Software apresentado à Faculdade de Computação da Universidade Federal de Uberlândia, Minas Gerais, como requisito exigido parcial à obtenção do grau de Bacharel em Sistemas de Informação.

Área de concentração: Sistemas de Informação

Orientador: Prof. Dr. Leonardo Muttoni

Uberlândia - MG
2025

Resumo

A principal motivação deste trabalho surge a partir das necessidades que existem em sistemas de busca em DOM's (Diários Oficiais Municipais). O Objetivo é desenvolver um sistema capaz de buscar arquivos PDF (Portable Document Format) de documentos disponibilizados por prefeituras, realizar a leitura e indexação do conteúdo destes arquivos e possuir um mecanismo facilitador de busca textual, retornando os documentos que sejam mais relevantes. Como base para o desenvolvimento foi utilizado o portal de diários da prefeitura de Uberlândia, onde a busca por estes documentos é limitada ao dia e mês de publicação. Para isso será desenvolvido um *Scraper* que fará o trabalho de obtenção, leitura e indexação, uma API (Application Programming Interface) que será responsável pelo acesso e busca no índice e servirá a uma interface web que permitirá aos usuários a busca, filtragem e acesso aos documentos.

Palavras-chave: DOM, Scraper, Índice.

Lista de ilustrações

Figura 1 – Cronograma de desenvolvimento 15
Figura 2 – Diagrama de caso de uso do sistema 15
Figura 3 – Protótipo inicial 21
Figura 4 – Versão inicial operável 22
Figura 5 – Versão final 22
Figura 6 – Arquitetura do Sistema de Recuperação 23
Figura 7 – Modelo de domínio do sistema 24
Figura 8 – Diagrama de sequência da busca 25
Figura 9 – Processo de Coleta e Indexação 26
Figura 10 – Processo de Busca 26
Figura 11 – Visualizacao dos Resultados 27
Figura 12 – Modelo Entidade Relacionamento 28
Figura 13 – Diagrama de classes da API backend 31
Figura 14 – Fluxos do sistema 32
Figura 15 – Diagrama de Implantação do Sistema 33
Figura 16 – Estrutura de Diretórios no servidor 45

Lista de tabelas

Tabela 1 – Custos de software 14
Tabela 2 – Custos de Infraestrutura 14
Tabela 3 – Tabela: docs 30
Tabela 4 – Tabela FTS5: docs_fts 30
Tabela 5 – Linguagens de desenvolvimento 38
Tabela 6 – Bibliotecas utilizadas 39
Tabela 7 – Sistemas e componentes externos 40
Tabela 8 – Componentes usados na execução 42
Tabela 9 – Variáveis de ambiente 46
Tabela 10 – Links de repositórios 51

Lista de siglas

Sumário

1 INTRODUÇÃO 9
1.1 Contextualização do trabalho 9
1.2 Problema e Justificativas 9
1.3 Objetivos 10
1.4 Contribuições Esperadas 10
1.5 Organização do Trabalho 11
1.6 IA-Generativa 11

2 REQUISITOS DO SISTEMA 12
2.1 Descrição geral do sistema 12
2.1.1 Abrangência e sistemas relacionados 12
2.1.2 Descrição dos usuários 13
2.2 Estudo de Viabilidade do Sistema 13
2.2.1 Benefícios Esperados 14
2.2.2 Custos do Sistema 14
2.2.3 Cronograma de Desenvolvimento 14
2.3 Requisitos funcionais - RF 15
2.3.1 Diagrama de Caso de Uso do sistema 15
2.3.2 Descrição dos Cenários dos Casos de Uso 16
2.4 Requisitos Não Funcionais - RNF 19
2.4.1 [RNF001] Desempenho 19
2.4.2 [RNF002] Usabilidade 19
2.4.3 [RNF003] Escalabilidade 20
2.4.4 [RNF004] Manutenabilidade 20
2.4.5 [RNF005] Disponibilidade 20
2.5 Protótipos das interfaces com usuários 21
2.6 Considerações Sobre o Capítulo 22

3 ANÁLISE E PROJETO DO SISTEMA 23
3.1 Modelo de Domínio do Sistema 24
3.2 Diagramas de Sequência do Sistema 24
3.2.1 Diagramas de Sequência - (RF001, RF002, RF003, RF004) - Processo de Coleta e Indexação 25
3.2.2 Diagramas de Sequência - (RF005, RF006, RF007) - Processo de Busca 25
3.2.3 Diagramas de Sequência - (RF008, RF009, RF010) - Visualização do Documento 27
3.3 Modelagem de Dados do Sistema 28
3.3.1 Modelo Lógico de Dados do Sistema 28
3.3.2 Modelo Físico de Dados do Sistema 29
3.3.3 Dicionário de Dados do Sistema 30
3.4 Diagrama de Classe do Sistema 30
3.5 Diagramas de Atividades 32
3.6 Diagrama de Implantação do Sistema 33
3.7 Considerações Sobre o Capítulo 34

4 DESENVOLVIMENTO DO SISTEMA 35
4.1 Processo de Desenvolvimento 35
4.2 Arquitetura do Sistema 36
4.3 Tecnologias Utilizadas no Desenvolvimento 37
4.3.1 Linguagens de desenvolvimento 38
4.4 Padrões de Projetos Utilizados 38
4.4.1 Bibliotecas 39
4.4.2 Ferramentas Auxiliares 40
4.4.3 Sistemas e componentes externos utilizados 40
4.5 Instalação e Configurações 40
4.5.1 Pré-requisitos do Sistema 41
4.5.2 Processo de Instalação/Execução 43
4.5.3 Variáveis de Ambientes Necessárias 46
4.5.4 Arquivos de Configuração 47
4.6 Utilização do Sistema 50
4.6.1 Processo de Execução do Sistema 50
4.6.2 Exemplos de Utilização das Principais Funcionalidades do Sistema . . . 50
4.7 Testes do Sistema 51
4.7.1 Plano de Testes 51
4.7.2 Execução do Plano de Testes <Opcional> 51
4.8 Repositório de Código 51
4.8.1 Link para o Repositório 51
4.8.2 Estrutura de Diretórios 51

4.8.3 Guia de Contribuições (se aplicável) 51
4.9 Resultados Alcançados com o Sistema 52
4.10 Considerações Sobre o Capítulo 52

5 CONCLUSÕES 53
5.1 Principais Contribuições 53
5.2 Trabalhos Futuros 53

REFERÊNCIAS 54

Capítulo 1

Introdução

1.1 Contextualização do trabalho

Os Diários Oficiais dos Municípios são instrumentos importantes de transparência pública, servindo para a divulgação de atos administrativos, licitações, nomeações, leis, decretos e outras informações diversas para o meio social. Apesar de sua importância, existem municípios brasileiros que disponibilizam seus Diários Oficiais por meio de plataformas com recursos limitados de consulta e busca, o que dificulta o acesso a informação e limita a transparência.

A Recuperação da Informação é a área da computação que cobre o armazenamento, organização e recuperação de documentos e conteúdos textuais. A área é de grande importância pois facilita o acesso a informação de maior interesse (BAEZA-YATES; RIBEIRO-NETO, 2013). O surgimento da *World Wide Web* impulsionou o crescimento da área pois se tornou um grande repositório de informação que precisa ser organizada para ser acessada. Tais técnicas de recuperação são amplamente utilizadas em mecanismos de busca e sistemas de gerenciamento de documentos. No contexto dos Diários Oficiais Municipais, a aplicação destas técnicas pode facilitar o acesso a informações de interesse público em formato digital.

Nesse sentido, este trabalho propõe o desenvolvimento de um sistema de indexação e busca de documentos voltado para diários oficiais municipais que permita que os usuários possam fazer buscas textuais de maneira facilitada.

1.2 Problema e Justificativas

A maioria dos DOMs é disponibilizada em arquivos PDF que são publicados diariamente, em alguns casos em formato de documento web com pouca organização, frequentemente sem mecanismos de indexação textual e organizados cronologicamente, com cada município escolhendo formas diferentes de distribuição dos arquivos. Isso torna a consulta lenta, manual e pouco intuitiva, sendo agravado quando se tem um volume maior

de documentos. Essa dificuldade afeta cidadãos, pesquisadores e outros interessados que necessitam consultar com regularidade as informações oficiais que são publicadas nos diários. Diante deste cenário surge a necessidade de soluções tecnológicas que tornem essas informações mais acessíveis e organizadas.

1.3 Objetivos

Um sistema de organização e recuperação de informação tem como composição um processo de coleta, indexação e recuperação (BAEZA-YATES; RIBEIRO-NETO, 2013) que permite que usuários tenham suas buscas por documentos facilitadas. Considerando este processo realizado por sistemas de organização e recuperação é possível traçar um paralelo com o objetivo deste trabalho, que é a criação de uma ferramenta que faça a coleta de documentos, indexação textual e busca em diários oficiais de municípios permitindo que os usuários localizem informações de forma estruturada e intuitiva.

A solução é composta por três módulos principais: um módulo responsável pela coleta e indexação dos documentos, um módulo de backend para processamento e acesso aos dados e um de frontend responsável pela interface de interação dos usuários. Este conjunto de módulos fazem a composição da ferramenta que permitirá que usuários realizem consultas rápidas e mais eficientes em documentos previamente carregados e processados.

Tendo em vista este objetivo, um rascunho da ideia será de desenvolver o sistema a partir dos seguintes módulos:

1. Identificar municípios que disponibilizam PDFs com texto extraível (2 municípios inicialmente).
2. Implementar um módulo robô *crawler* que faz o download dos arquivos PDF.
3. Processar estes arquivos, realizando a indexação de conteúdo, extraindo seus textos, relacionando trechos às páginas, datas e URLs de origem.
4. Construir um backend web capaz de realizar consultas ao índice textual e disponibilizar endpoints via API REST.
5. Construir um frontend web que permite buscas e apresente os resultados.
6. Aplicar testes e coletar feedback sobre a ferramenta.

1.4 Contribuições Esperadas

A transparência e o acesso à informação constituem os principais pilares deste trabalho, a lei de acesso à informação (BRASIL, 2011) determina que dados públicos devem ser

oferecidos de forma clara e acessível. Entretanto, na prática, a dificuldade de realizar buscas limita a transparência e este acesso a informação.

A carência de ferramentas do tipo é notável, visto que cada município tem formas diferentes de acesso, alguns possuem ferramentas de maior facilidade de uso, como no portal de acesso da prefeitura do município de Patos de Minas - MG (MINAS, 2025) enquanto outras prefeituras utilizam ferramentas terceiras que reúnem os documentos de muitos municípios e não são tão intuitivas.

Uma ferramenta de busca estruturada pode beneficiar jornalistas, advogados, pesquisadores, servidores e cidadãos em geral, a automação do processo de busca de documentos reduz o tempo de consulta, amplia o acesso e moderniza o serviço público.

1.5 Organização do Trabalho

O presente trabalho está dividido da seguinte maneira: O capítulo 1 apresenta a introdução e descrição do trabalho, no capitulo 2 são apresentados os requisitos do sistema

1.6 IA-Generativa

A IA-Generativa foi utilizada para auxiliar na correção de bugs das ferramentas desenvolvidas neste trabalho, além da geração de código para diagramas UML que são apresentados nesta monografia, criados a partir da ferramenta plantUML.

Capítulo 2

Requisitos do Sistema

Neste capítulo é descrito o levantamento de requisitos do sistema de indexação e busca em diários oficiais dos municípios.

2.1 Descrição geral do sistema

O sistema proposto tem o objetivo de permitir que usuários possam realizar busca em documentos de diários oficiais municipais que são disponibilizados por diferentes prefeituras.

A arquitetura é composta por três camadas principais: coleta e indexação, backend e interface web. O módulo de coleta desenvolvido em python faz o trabalho de scraping no site da prefeitura e realiza a indexação dos dados dos documentos PDF dos diários oficiais obtidos em uma base de dados SQLite que permite busca textual através o mecanismo FTS (Full-Text Search).

A camada backend é composta por uma API REST desenvolvida em .NET, responsável por consultar o banco de dados indexado pela ferramenta de coleta e devolver os resultados de busca. A terceira camada é uma aplicação web desenvolvida em React que atuará como interface de usuário, permitindo a busca e filtragem dos documentos indexados.

2.1.1 Abrangência e sistemas relacionados

O sistema é pensado para funcionar de forma modular, sendo que cada um dos 3 módulos principais (scraping e indexação, backend e frontend) tem papeis bem definidos e operam entre si. O módulo de scraping e indexação pode trabalhar de forma a receber como entrada comandos para realizar o download (através do site da prefeitura) e indexação de documentos de uma cidade e mês específicos ou indexar documentos já presentes no sistema de arquivos.

Já a API REST do módulo de backend fica responsável por receber as consultas textuais e filtragens, que então serão realizadas no banco de dados indexado e retornará

os resultados obtidos. O módulo de frontend ficará responsável por fornecer ao usuário uma interface visual que lhe permita realizar as consultas e filtragens, além de permitir a pré-visualização e seleção dos documentos recuperados.

2.1.2 Descrição dos usuários

O sistema desenvolvido é voltado a usuários que necessitam consultar informações publicadas em diários oficiais municipais de forma rápida e organizada. De modo geral, os usuários possuem familiaridade com navegação na web.

O principal problema enfrentado é que os portais de acesso aos DOM's não são padronizados e algumas prefeituras não oferecem um serviço de busca textual aos documentos, dificultando a consulta das informações publicadas nos diários.

2.1.2.1 Cidadãos

Cidadãos constituem um dos principais grupos de usuários. Este grupo inclui moradores do município, contribuintes e demais interessados em acompanhar os atos administrativos.

2.1.2.2 Jornalistas e pesquisadores

Jornalistas e pesquisadores utilizam diários oficiais como fontes de informação para investigação, produção de reportagens e estudos diversos. Este grupo de usuários pode precisar analisar um grande volume de documentos, além de encontrar termos específicos, nomes, empresas ou processos administrativos. O que acaba sendo dificultado em casos em que a busca e filtragem de documentos não é possível.

2.1.2.3 Servidores Públicos e Profissionais do Direito

Esses usuários tem maior familiaridade com documentos oficiais, normalmente fazendo o acesso a estes documentos para acompanhamento de processos administrativos, prazos e atos normativos. A falta de mecanismos para busca textual também dificulta que eles possam encontrar informação que seja relevante em suas consultas.

2.2 Estudo de Viabilidade do Sistema

Por se tratar de basicamente uma interface web de busca, a viabilidade de desenvolvimento se mostra bastante facilitada com o uso de ferramentas modernas.

É necessário um servidor web para a disponibilização do site e da API backend, além de espaço de armazenamento para o banco de dados SQLite que guarda os dados dos arquivos indexados. A execução do script de indexação pode ser feita manualmente ou por algum recurso de agendamento automatizado.

Pensando no uso em um cenário real, o módulo de scraping passa a funcionar apenas como módulo de indexação, já que uma prefeitura que fizesse o uso da ferramenta já teria o acesso aos documentos que só precisariam passar pela indexação no banco de dados para serem disponibilizados para busca textual.

2.2.1 Benefícios Esperados

O sistema permite que diferentes prefeituras possam utilizar a ferramenta para disponibilizar aos usuários uma interface amigável e mais organizada para que possam realizar buscas textuais e filtragens dentro dos DOM‘s disponibilizados. Destacando a redução de tempo necessário para localizar informações nos documentos e a melhoria de transparência de informação pública .

2.2.2 Custos do Sistema

Estimativas preliminares de orçamento para o projeto:

Tabela 1 – Custos de software

Recurso	Custo
Python	Gratuito
SQLite	Gratuito
.NET	Gratuito
React	Gratuito
Nginx	Gratuito

Fonte: O autor (2026).

Tabela 2 – Custos de Infraestrutura

Recurso	Custo
Hospedagem VPS	R$ 20 a R$ 50/mês
Domínio	R$ 40/ano

Fonte: O autor (2026).

2.2.3 Cronograma de Desenvolvimento

O cronograma geral de desenvolvimento dos módulos do sistema pode ser visualizado na figura 1:

A ideia do cronograma de desenvolvimento foi seguir a divisão de módulos do sistema, como o módulo de scraping e indexação precisa funcionar antes do processo de busca, o desenvolvimento partiu dele. Este módulo exigiu mais tempo de desenvolvimento pois foi necessário um período de entendimento de como a ferramenta deveria operar e quais ferramentas e/ou bibliotecas poderiam ser utilizadas para atingir o objetivo de obter e

	nov/25				dez/25				fev/26				mar/26				mai/26				jun/26			
semana	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
DEV: Scraper																								
DEV: Backend																								
DEV: Frontend																								
Refinamento de código																								

Figura 1 – Cronograma de desenvolvimento

indexar os documentos de forma satisfatória. Além do tempo adicional para adaptar a ferramenta a realizar o scraping especificamente ao site da prefeitura de Uberlândia, que foi usado como base para a construção da ferramenta. A base para o desenvolvimento do sistema de busca foi construída no primeiro módulo, o que facilitou o desenvolvimento dos módulos seguintes, já que eles funcionam como consumidores do trabalho realizado pelo primeiro módulo e possuem baixa complexidade, tanto na API quanto no módulo de frontend, estes exigiram menor tempo de desenvolvimento.

2.3 Requisitos funcionais - RF

2.3.1 Diagrama de Caso de Uso do sistema

O diagrama de casos de uso abaixo apresenta as principais funcionalidades do sistema de busca em Diários Oficiais.

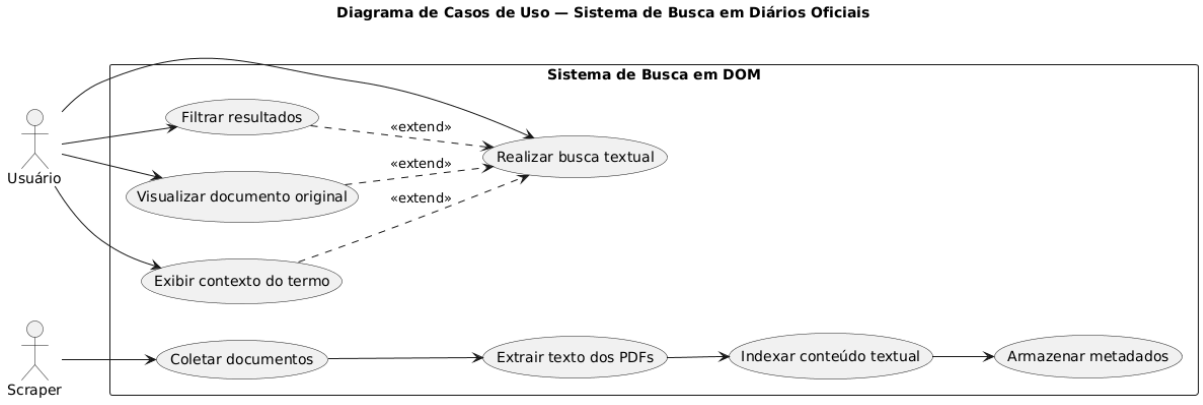


Figura 2 – Diagrama de caso de uso do sistema

2.3.2 Descrição dos Cenários dos Casos de Uso

2.3.2.1 [RF001] Realizar coleta de documentos

- ❑ **Descrição:** O sistema deve ser capaz de acessar o site da prefeitura e realizar o download dos arquivos PDF do Diário Oficial disponíveis.
- ❑ **Ator:** Módulo de scraping
- ❑ **Prioridade:** Essencial
- ❑ **Entradas e pré condições:** O site da prefeitura deve estar acessível e/ou o diretório contendo os arquivos para indexação.
- ❑ **Saídas e pós condições:** Base de dados atualizada.
- ❑ **Fluxo de eventos principal**

2.3.2.2 [RF002] Extrair texto dos documentos

- ❑ **Descrição:** O sistema deve extrair o conteúdo textual dos arquivos PDF coletados.
- ❑ **Ator:** Módulo de scraping
- ❑ **Prioridade:** Essencial
- ❑ **Entradas e pré condições:** Documentos devem estar acessíveis para leitura e estruturados de forma que possibilite a extração textual.
- ❑ **Saídas e pós condições:** Base de dados atualizada.
- ❑ **Fluxo de eventos principal**
- ❑ **Fluxos secundários** (alternativos e de exceção)

2.3.2.3 [RF003] Indexar o conteúdo textual

- ❑ **Descrição:** O sistema deve armazenar e indexar o texto extraído utilizando um mecanismo de busca textual.
- ❑ **Ator:** Módulo de scraping
- ❑ **Prioridade:** Essencial
- ❑ **Entradas e pré condições:** O conteúdo textual do documento deve ser obtido.
- ❑ **Saídas e pós condições:** Base de dados atualizada.
- ❑ **Fluxo de eventos principal**
- ❑ **Fluxos secundários** (alternativos e de exceção)

2.3.2.4 [RF004] Armazenar metadados dos documentos

- **Descrição:** O sistema deve armazenar informações associadas aos documentos, como data, nome, número de página e origem do documento.
- **Ator:** Módulo de scraping
- **Prioridade:** Essencial
- **Entradas e pré condições:** O nome e data do documento devem ser obtidos.
- **Saídas e pós condições:** Base de dados atualizada.
- **Fluxo de eventos principal**
- **Fluxos secundários** (alternativos e de exceção)

2.3.2.5 [RF005] Disponibilizar API de consulta

- **Descrição:** O sistema deve disponibilizar uma API REST que permita a consulta dos documentos indexados.
- **Ator:** Módulo de backend
- **Prioridade:** Essencial
- **Entradas e pré condições:** A base de dados indexada deve estar acessível.
- **Saídas e pós condições:** Serviço de consulta disponível.
- **Fluxo de eventos principal**
- **Fluxos secundários** (alternativos e de exceção)

2.3.2.6 [RF006] Realizar busca textual

- **Descrição:** O sistema deve permitir a busca por palavras-chave no conteúdo dos documentos indexados.
- **Ator:** Módulo de backend
- **Prioridade:** Essencial
- **Entradas e pré condições:** A base de dados indexada deve estar acessível e as requisições devem estar corretamente formatadas.
- **Saídas e pós condições:** Listagem de documentos que correspondem a busca realizada.

- ❑ **Fluxo de eventos principal**
- ❑ **Fluxos secundários** (alternativos e de exceção)

2.3.2.7 [RF007] Filtrar resultados de busca

- ❑ **Descrição:** O sistema deve permitir a aplicação de filtros nos resultados de busca, como data, período e edição.
- ❑ **Ator:** Módulo de backend
- ❑ **Prioridade:** Essencial
- ❑ **Entradas e pré condições:** A base de dados indexada deve estar acessível e as requisições devem estar corretamente formatadas.
- ❑ **Saídas e pós condições:** Listagem de documentos que correspondem a filtragem realizada.
- ❑ **Fluxo de eventos principal**
- ❑ **Fluxos secundários** (alternativos e de exceção)

2.3.2.8 [RF008] Exibir resultados da busca

- ❑ **Descrição:** O sistema deve apresentar ao usuário os documentos encontrados contendo o termo pesquisado.
- ❑ **Ator:** Módulo de frontend
- ❑ **Prioridade:** Essencial
- ❑ **Entradas e pré condições:** A API de busca deve estar acessível.
- ❑ **Saídas e pós condições:** Listagem visual de documentos que correspondem a filtragem realizada.
- ❑ **Fluxo de eventos principal**
- ❑ **Fluxos secundários** (alternativos e de exceção)

2.3.2.9 [RF009] Exibir contexto do resultado

- ❑ **Descrição:** O sistema deve mostrar um trecho do texto em que o termo pesquisado aparece.
- ❑ **Ator:** Módulo de frontend

- ❏ **Prioridade:** Importante
- ❏ **Entradas e pré condições:** A API de busca deve estar acessível.
- ❏ **Saídas e pós condições:** Listagem textual de conteúdos de documentos que correspondem a busca realizada.
- ❏ **Fluxo de eventos principal**
- ❏ **Fluxos secundários** (alternativos e de exceção)

2.3.2.10 [RF010] Permitir acesso ao documento original

- ❏ **Descrição:** O sistema deve permitir que o usuário acesse ou visualize o documento PDF original correspondente ao resultado da busca.
- ❏ **Ator:** Módulo de frontend
- ❏ **Prioridade:** Essencial
- ❏ **Entradas e pré condições:** Os documentos devem estar disponíveis no serviço de hospedagem.
- ❏ **Saídas e pós condições:** Acesso aos documentos recuperados pela busca
- ❏ **Fluxo de eventos principal**
- ❏ **Fluxos secundários** (alternativos e de exceção)

2.4 Requisitos Não Funcionais - RNF

2.4.1 [RNF001] Desempenho

- ❏ **Descrição:** O sistema deve retornar os resultados das consultas em tempo adequado, permitindo buscas eficientes mesmo com grande quantidade de documentos indexados.
- ❏ **Prioridade:** Importante.
- ❏ **Caso(s) de uso associado(s):** RF006, RF008

2.4.2 [RNF002] Usabilidade

- ❏ **Descrição:** A interface web deve ser simples e intuitiva, permitindo que usuários realizem buscas sem necessidade de treinamento prévio.
- ❏ **Prioridade:** Importante.
- ❏ **Caso(s) de uso associado(s):** RF006, RF008, RF009, RF010

2.4.3 [RNF003] Escalabilidade

- ❑ **Descrição:** A arquitetura do sistema deve permitir a expansão para indexar documentos de outras prefeituras ou fontes de Diários Oficiais.
- ❑ **Prioridade:** Desejável.
- ❑ **Caso(s) de uso associado(s):** RF001, RF003, RF004, RF005

2.4.4 [RNF004] Manutenabilidade

- ❑ **Descrição:** O sistema deve possuir uma arquitetura modular que facilite manutenção, atualização e inclusão de novas funcionalidades.
- ❑ **Prioridade:** Desejável.
- ❑ **Caso(s) de uso associado(s):**

2.4.5 [RNF005] Disponibilidade

- ❑ **Descrição:** A API e a interface web devem estar disponíveis para acesso sempre que o sistema estiver em operação.
- ❑ **Prioridade:** Essencial.
- ❑ **Caso(s) de uso associado(s):** RF005, RF006, RF007, RF008, RF009, RF010

2.4.5.1 [RNF006] Compatibilidade

- ❑ **Descrição:** A aplicação web deve ser compatível com os principais navegadores modernos.
- ❑ **Prioridade:** Essencial
- ❑ **Caso(s) de uso associado(s):** RF008, RF009, RF010

2.4.5.2 [RNF007] Portabilidade

- ❑ **Descrição:** O sistema deve ser capaz de ser executado em diferentes ambientes computacionais compatíveis com as tecnologias utilizadas.
- ❑ **Prioridade:** Importante.
- ❑ **Caso(s) de uso associado(s):** RF010

2.4.5.3 [RNF008] Confiabilidade

- ❏ **Descrição:** O sistema deve garantir a integridade dos dados armazenados no banco de dados.
- ❏ **Prioridade:** Essencial.
- ❏ **Caso(s) de uso associado(s):** RF010

2.5 Protótipos das interfaces com usuários

A figura 3 ilustra um esboço do layout da pagina de buscas, a ideia foi manter todo o conteúdo necessário em somente uma página, com o componente de formulário de pesquisa sendo o elemento central. Além de uma exibição do documento mais recente.

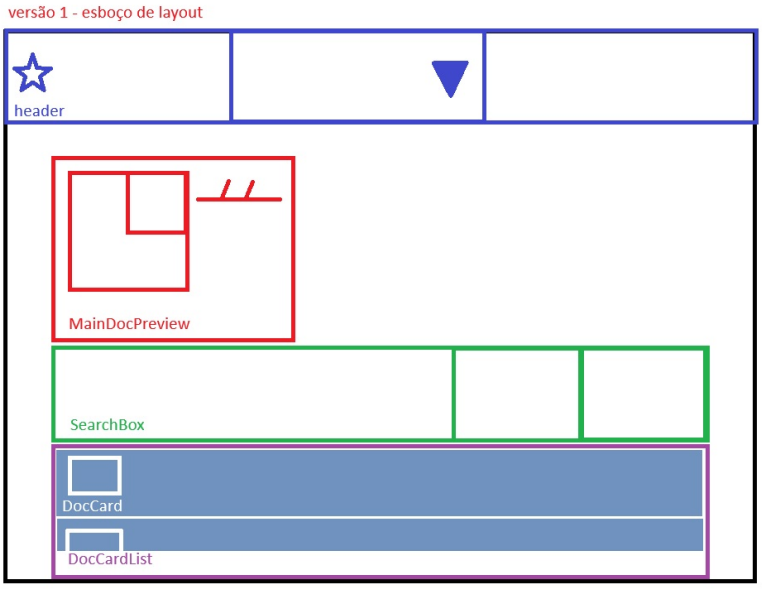


Figura 3 – Protótipo inicial

Já a figura 4 exibe uma captura da tela em estágio de desenvolvimento, a estrutura segue o que foi definido no esboço inicial e serviu como ponto de partida para refinamento final. Neste estágio o sistema já estava operacional para realização de buscas e exibição de resultados, porém sua estrutura visual ainda não estava preparada para ter responsividade para o acesso através de dispositivos móveis.

Por fim, na captura da figura 5 contém o sistema em estágios finais de desenvolvimento, com refinamentos na exibição de resultados e com responsividade melhorada para permitir o acesso da página através de dispositivos móveis.

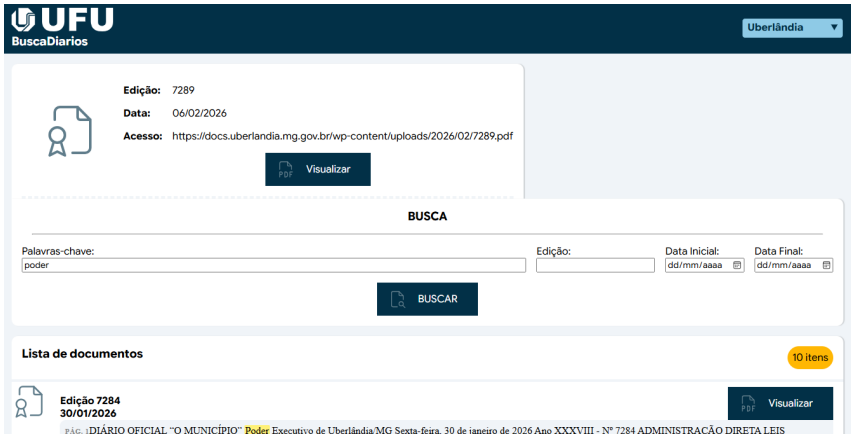


Figura 4 – Versão inicial operável

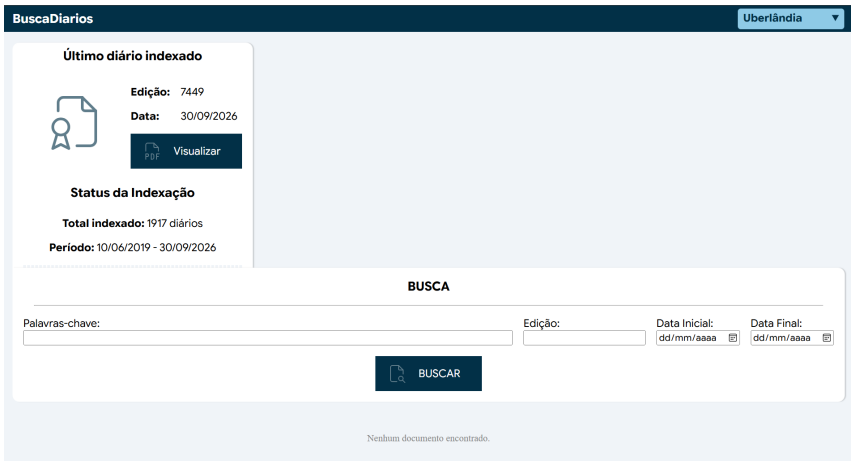


Figura 5 – Versão final

2.6 Considerações Sobre o Capítulo

Neste capítulo foram apresentados os requisitos do sistema de busca em Diários Oficiais Municipais, incluindo a arquitetura geral da solução, a caracterização dos usuários, os requisitos funcionais e não funcionais e os casos de uso identificados. A especificação apresentada estabelece uma base para o desenvolvimento e avaliação da solução proposta, permitindo que as etapas de implementação sejam conduzidas de forma estruturada e alinhada aos objetivos do trabalho

Capítulo 3

Análise e Projeto do Sistema

Este capítulo apresenta a análise e projeto do sistema de acordo com os requisitos levantados, a figura 6 apresenta uma visualização da arquitetura planejada, demonstrando como seus módulos e atores interagem.

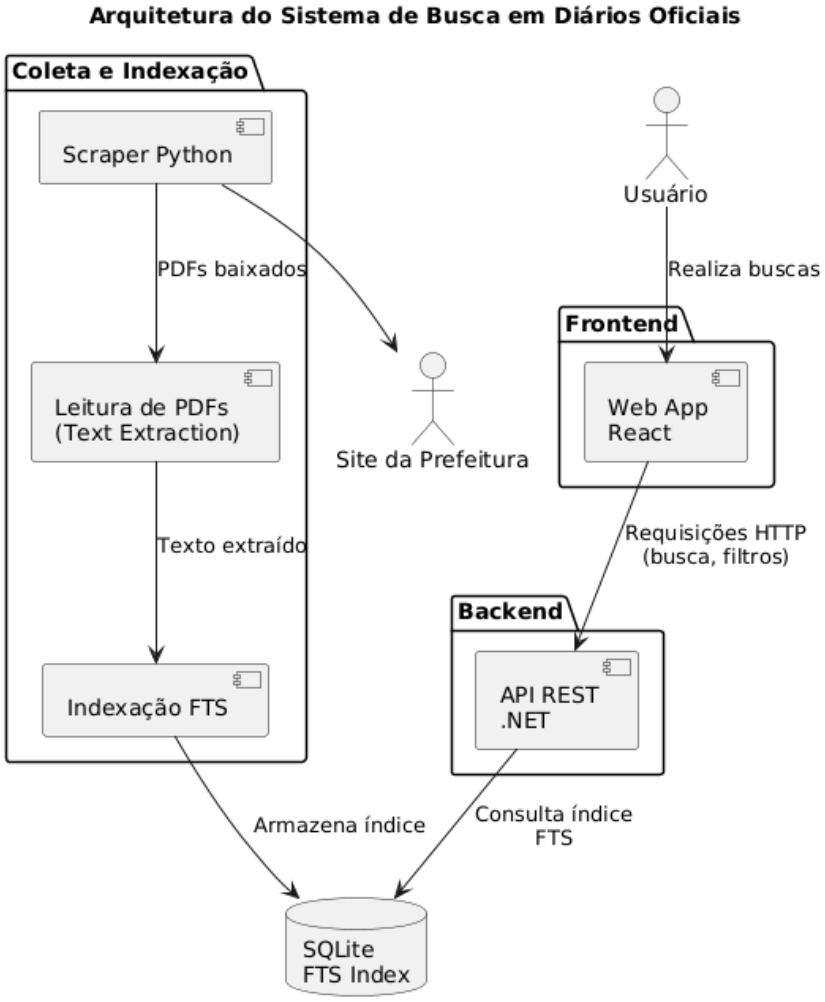


Figura 6 – Arquitetura do Sistema de Recuperação

3.1 Modelo de Domínio do Sistema

O modelo inicial de domínio da figura 7 representa o problema que o sistema busca resolver, demonstrando o fluxo conceitual: uma cidade publica vários diários que são compostos por várias páginas, as páginas são indexadas em um índice textual que é consultado conforme os critérios de busca do usuário para que os documentos sejam recuperados e exibidos de acordo com estes critérios.

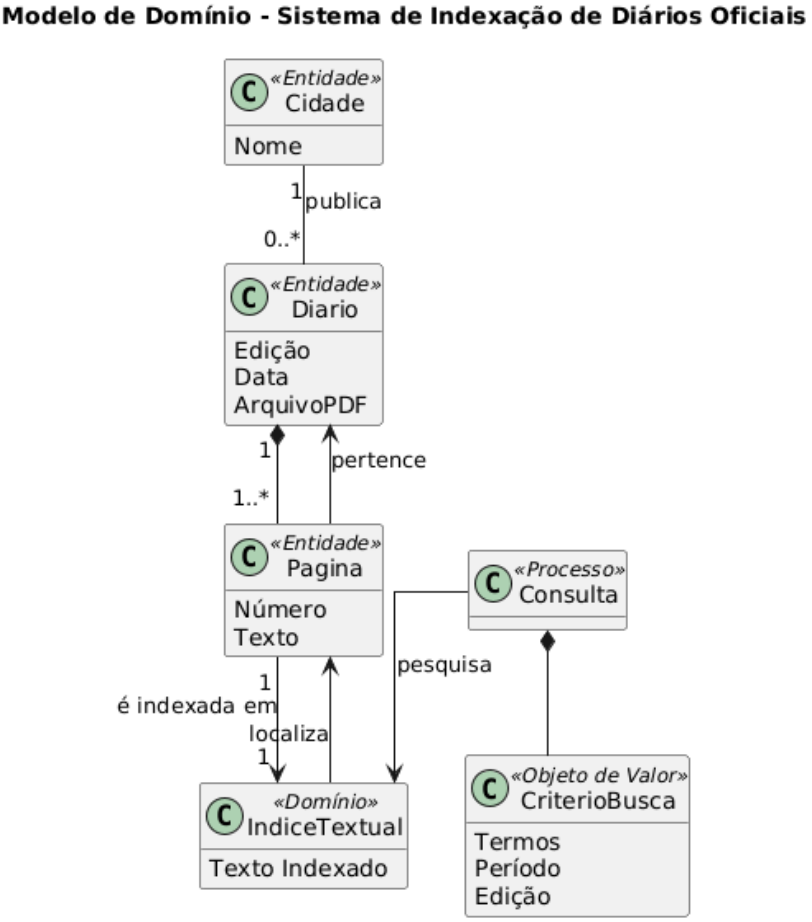


Figura 7 – Modelo de domínio do sistema

A partir deste modelo já é possível ter uma ideia inicial dos módulos que compõem o sistema. Se faz necessário um componente de obtenção e indexação dos documentos, um componente responsável pelas consultas e um componente de exibição dos resultados, no caso deste trabalho, como mostrado no modelo de arquitetura da figura 6, o paralelo são os módulos de coleta e indexação, backend e frontend, respectivamente.

3.2 Diagramas de Sequência do Sistema

A seguir são apresentados os diagramas de sequência mais relevantes do sistema. A figura 8 apresenta o diagrama criado inicialmente para representar a sequência da busca

textual, partindo da interação do usuário com o frontend enviando uma requisição de busca nos diários indexados e retorno da resposta pela API.

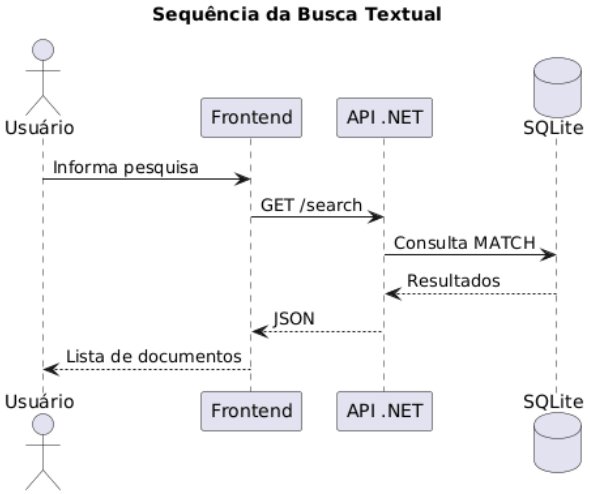


Figura 8 – Diagrama de sequência da busca

Nos itens seguintes são apresentados os diagramas referentes ao processo de coleta e indexação, processo de busca e processo de visualização dos documentos, assim como os requisitos funcionais correspondentes a estes processos.

3.2.1 Diagramas de Sequência - (RF001, RF002, RF003, RF004) - Processo de Coleta e Indexação

O diagrama do processo de coleta e indexação (figura 9) abrange os requisitos funcionais RF001, RF002, RF003 e RF004, nele é descrito o processo realizado para coleta e indexação dos documentos, sendo um operador manual ou ferramenta de agendamento de execução que inicia o processo de execução da ferramenta, que cuida da obtenção da lista de documentos, download e indexação no banco SQLite.

A indexação é feita a partir da leitura de cada página do documento separadamente, que salva na tabela "docs_fts"o número de página, conteúdo e id de documento correspondente. Após a indexação total das páginas do documento o processo é concluído.

3.2.2 Diagramas de Sequência - (RF005, RF006, RF007) - Processo de Busca

Já o diagrama de busca (figura 10) abrange os requisitos funcionais RF005, RF006 e RF007, nele é descrito o processo de busca desde a requisição feita pelo usuário no frontend web, passando pelos módulos de backend até o retorno da lista de documentos obtidos de acordo com a pesquisa realizada. Neste processo o usuário define termos de

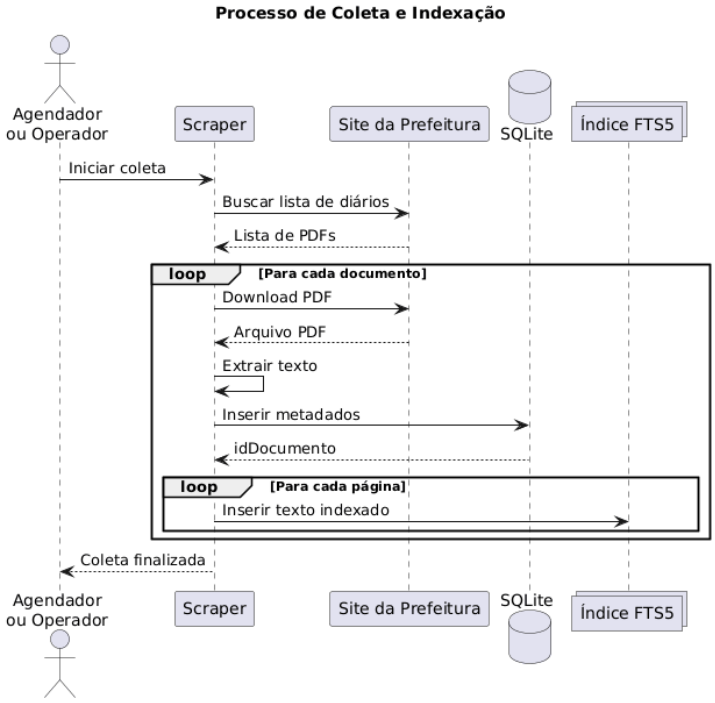


Figura 9 – Processo de Coleta e Indexação

busca, filtragem por datas e edição e a partir disto a API REST monta a consulta que será realizada ao banco de dados.

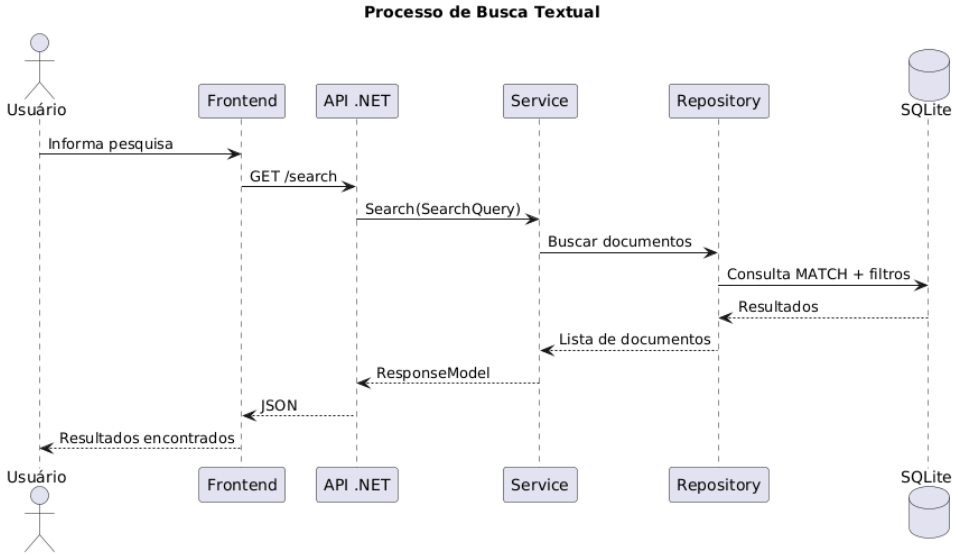


Figura 10 – Processo de Busca

A API possui um endpoint de pesquisa ("/search"), que recebe a requisição com os dados de busca definidos pelo usuário e a montagem da consulta é feita a depender de quais dados de busca foram enviados na requisição, sendo possível buscar apenas por consulta de texto, datas e edição, ou o conjunto destas informações. O resultado da

consulta realizada é serializado em formato JSON e devolvido ao frontend para exibição.

3.2.3 Diagramas de Sequência - (RF008, RF009, RF010) - Visualização do Documento

O diagrama de visualização da figura 11 expande o diagrama do processo de busca da figura 10, oferecendo uma visão de como é estruturada a busca pelos documentos que o usuário pretende visualizar.

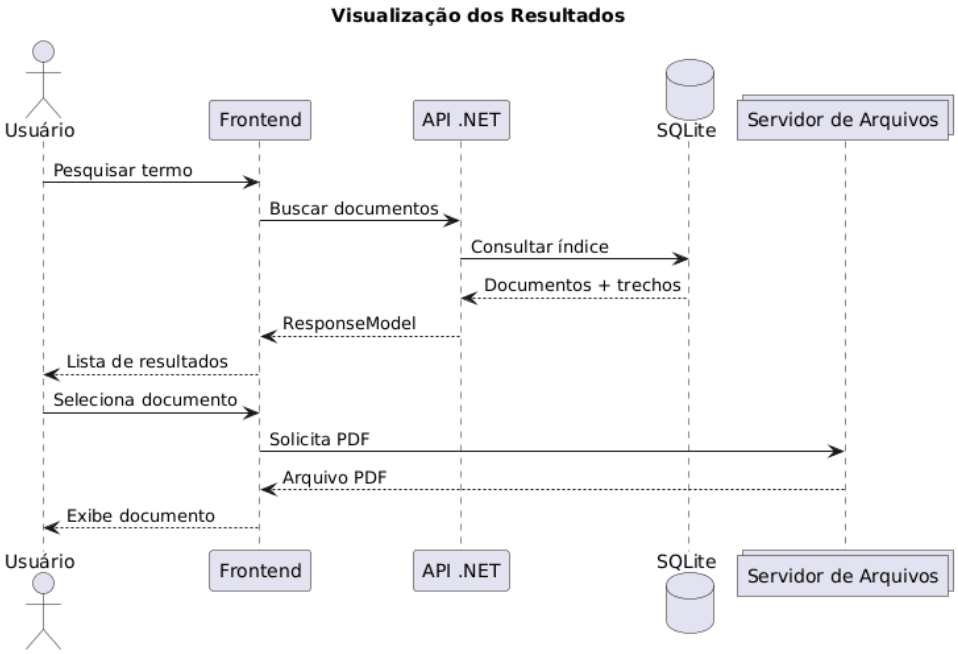


Figura 11 – Visualizacao dos Resultados

Depois que o usuário definiu quais termos de busca e filtros serão aplicados, esta informação é repassada para a API que realiza a busca na base de dados usando a consulta definida. Primeiramente é obtida a lista de id's de documentos que resultam da busca realizada, a partir destes id's são obtidos os metadados e as 3 primeiras páginas de cada um destes documentos que contém os trechos pesquisados (no caso de busca textual), caso a busca seja feita somente por data a API retorna somente a primeira página do documento. O frontend recebe a resposta da consulta com estes dados e faz a listagem usando as informações de edição, data e link de acesso ao pdf, além de exibir a prévia das páginas que contém os trechos pesquisados na listagem.

3.3 Modelagem de Dados do Sistema

3.3.1 Modelo Lógico de Dados do Sistema

A figura 12 apresenta o modelo de Entidade-Relacionamento construído nas versões iniciais do desenvolvimento e o modelo final após o desenvolvimento do sistema. Como o SQLite não possui um tipo de dado específico para armazenar datas, apenas funções de tempo que realizam operações usando tipos numéricos e de texto, a ideia inicial foi armazenar os valores de ano, mês e dia de forma separada em campos com tipos de dados numéricos.

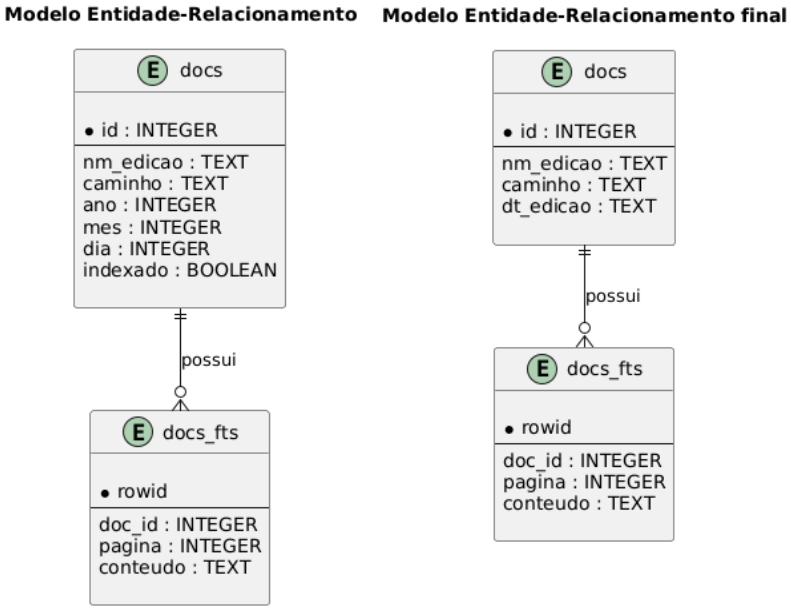


Figura 12 – Modelo Entidade Relacionamento

Com o avanço do desenvolvimento, este modelo foi alterado para que estes valores fossem armazenados em um único campo de texto, utilizando o formato YYYY-MM-DD (ISO-8601, 2019) para representar a data, o que permite que a tabela fique mais simplificada e as consultas que dependem de filtragem por tempo também sejam melhoradas. Além disso, o formato armazenado é reconhecido por diferentes linguagens de programação que possuem funções que trabalham com datas, como por exemplo C# usado na API backend que possui as classes "DateTime"e "DateOnly". O campo "indexado"também foi removido já que os documentos só aparecem no banco depois de serem indexados, o que tornaria o campo redundante.

Diferente de outros motores de banco de dados que normalmente precisam de um trabalho de instalação e configuração maior, o SQLite funciona através de um único arquivo, isto permitiu um nível maior de flexibilidade ao lidar com a base de dados. Este projeto foi estruturado de forma que para cada nova cidade cadastrada um novo arquivo de base de dados é criado, ou seja, a separação entre os documentos indexados de

diferentes cidades é feita por arquivo de base de dados SQLite. Para realizar a busca e/ou indexação é necessário que a flag que corresponde a cidade seja passada para a API (em casos de consulta) e para o scraper (em caso de execução de indexação), esta flag precisa ser adicionada nos arquivos de configuração dos dois módulos para que as ferramentas possam operar sobre a nova base de dados.

3.3.2 Modelo Físico de Dados do Sistema

Os scripts abaixo representam a criação de tabelas do banco, já utilizando o modelo final definido. O código 3.1 apresenta o script de criação da tabela "docs", que contém os metadados do documento, já o código 3.2 apresenta o script de criação da tabela "docs_fts", que armazena as páginas indexadas de cada documento.

Código 3.1 – Criação da tabela de documentos

```
1 CREATE TABLE IF NOT EXISTS docs(
2     id INTEGER PRIMARY KEY AUTOINCREMENT,
3     nm_edicao TEXT UNIQUE NOT NULL,
4     caminho TEXT NOT NULL,
5     dt_edicao TEXT NOT NULL);
```

Fonte: O autor (2026).

A tabela "docs"apresentada no código 3.1 serve como um indice dos documentos que foram lidos e indexados, ela contém uma chave de id único para cada documento e seus dados básicos que são utilizados pelo frontend para listagem, além do link de acesso direto ao arquivo pdf do diário, armazenado na coluna "caminho".

Código 3.2 – Criação da tabela de indexação dos documentos

```
1 CREATE VIRTUAL TABLE IF NOT EXISTS docs_fts
2 USING fts5(doc_id, pagina, conteudo, tokenize = "unicode61
     remove_diacritics 0");
```

Fonte: O autor (2026).

No caso do código 3.2, referente a criação da tabela de indexação de documentos, além das colunas definidas, também é passado para a função fts5 o parâmetro "tokenize"que permite escolher qual algoritmo será utilizado no processo de indexação textual na tabela, além de propriedades para o algoritmo. No caso deste trabalho foi utilizada a propriedade "remove_diacritics 0"junto com o algoritmo unicode61, esta propriedade serve para que o algoritmo faça diferenciação de caracteres como "ç"e "c", caso a propriedade não seja definida o algoritmo faz o processo de "tokenização"destes caracteres de forma a considera-los como o mesmo caractere, o que pode refletir em resultados de busca indesejados.

3.3.3 Dicionário de Dados do Sistema

O banco de dados do sistema foi modelado para armazenar os metadados e o conteúdo dos documentos indexados, permitindo que as consultas textuais sejam feitas utilizando o mecanismo FTS5 do SQLite. Isso foi feito através de duas tabelas, a primeira é a tabela "docs"que armazena os dados dos documentos obtidos e funciona como principal entidade do sistema.

Tabela 3 – Tabela: docs

Campo	Tipo	Chave	Descrição
id	INTEGER	PK	Chave de identificação do documento indexado
nm_edicao	TEXT		Campo que contém a edição (ex: 7389-A) do diário oficial
caminho	TEXT		Campo que contém o link de acesso ao arquivo indexado
dt_edicao	TEXT		Campo que contém a data do diário oficial em formato YYYY-MM-DD

Fonte: O autor (2026).

A segunda é a tabela "docs_fts", que é uma tabela criada utilizando o mecanismo FTS5 que serve como tabela de indexação do conteúdo dos documentos. É através dela que o recurso de busca textual do SQLite opera.

Tabela 4 – Tabela FTS5: docs_fts

Campo	Tipo	Chave	Descrição
doc_id	INTEGER	FK	Campo usado como chave estrangeira que aponta para a chave primaria da tabela docs
pagina	INTEGER		Campo que armazena o número da página indexada
conteudo	TEXT		Campo que armazena o texto extraído da página

Fonte: O autor (2026).

Todo o conteúdo dos documentos é separado por páginas durante a leitura do arquivo no módulo de scraping e cada página é armazenada separadamente na tabela. No SQLite, internamente todo conteúdo das tabelas criadas usando este mecanismo é tratado e otimizado como texto, por isso suas colunas não possuem tipo como foi possível visualizar anteriormente no código 3.2, os dados armazenados são parte de um índice invertido e são "tokenizados"para otimização de busca textual.

3.4 Diagrama de Classe do Sistema

A figura 13 contém o diagrama de classes da API, que apresenta as classes de modelos de requisição, resposta e entidade do sistema.

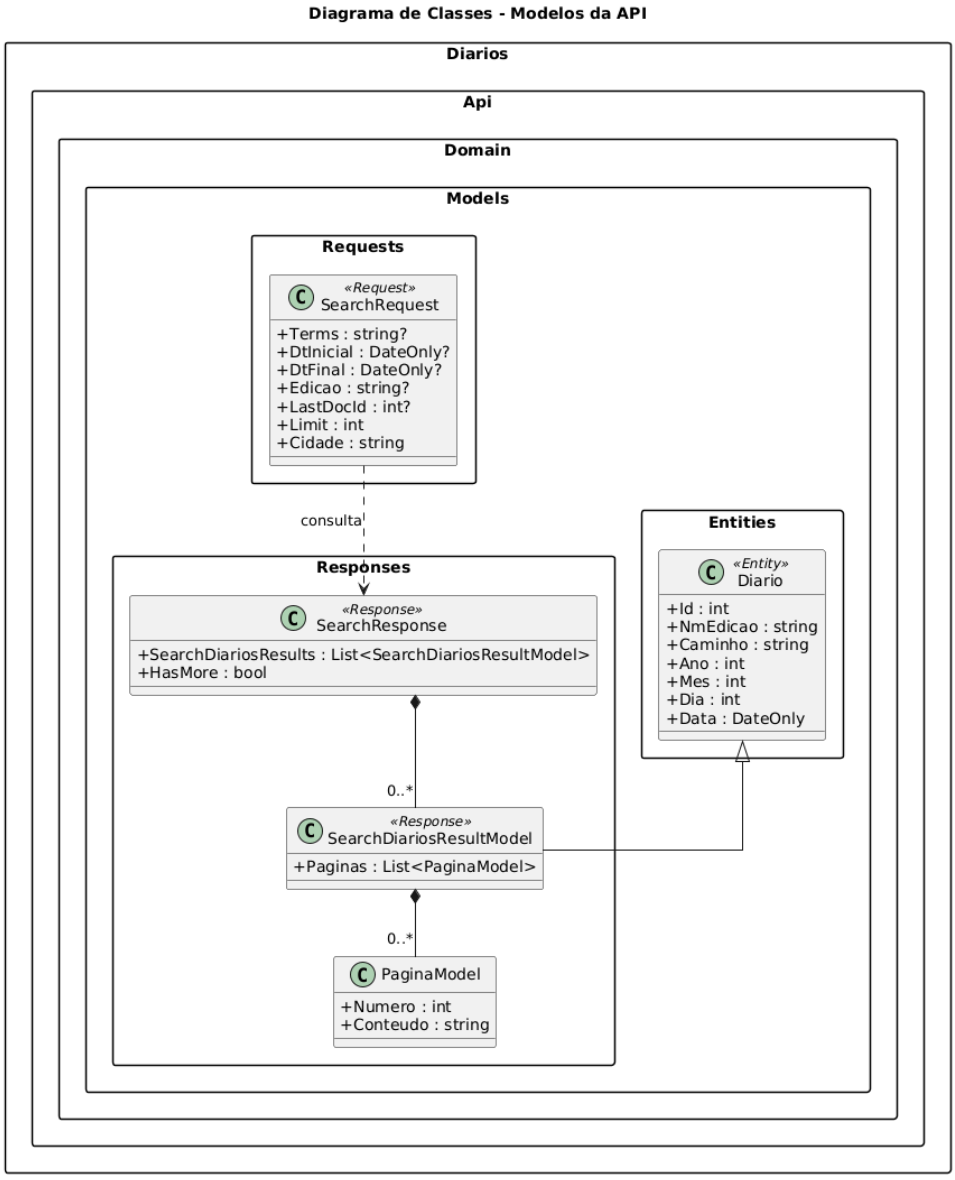


Figura 13 – Diagrama de classes da API backend

O pacote de requests contém a classe "SearchRequest"que é o modelo de dados que a API espera receber para realizar a busca, nela está definido os campos de termos da busca, datas para a filtragem, edição, ultimo id de documento buscado, limite de documentos na resposta e cidade correspondente. No pacote de Responses a classe "SearchResponse"representa o modelo de retorno da busca, contendo uma lista de resultados encontrados, sendo estes os diários e o conteúdo das páginas que correspondem com os termos pesquisados para exibição de prévia no frontend.

3.5 Diagramas de Atividades

A figura 14 demonstra o diagrama de fluxo de busca do sistema, que descreve o caminho percorrido desde a solicitação de busca pelo usuário até a exibição dos resultados e o diagrama de fluxo de coleta e indexação, que descreve o fluxo de obtenção dos documentos até o fim do processo de indexação.

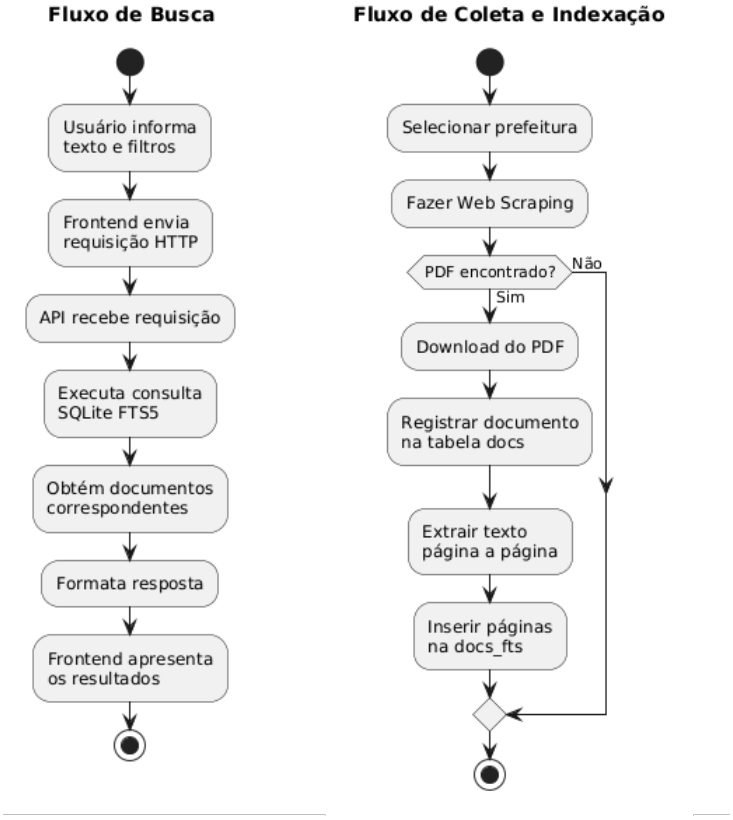


Figura 14 – Fluxos do sistema

No fluxo de coleta e indexação, o software foi pensado para receber uma lista de parâmetros (flags) para dar início ao seu fluxo de execução, neste trabalho foram desenvolvidos dois fluxos, o principal, que faz a busca e indexação dos documentos presentes no site da prefeitura de Uberlândia, neste caso é informado ao programa as flags de download, cidade, ano e mês. A execução de busca no site da prefeitura, download e indexação dos documentos e seus metadados é realizada automaticamente. O segundo fluxo é o de indexação local, onde é possível utilizar arquivos salvos localmente para serem indexados, neste caso é necessário informar o documento, a cidade, URI de acesso ao documento, além do ano, mês e dia correspondente ao diário sendo indexado.

Já no fluxo de busca, o primeiro passo é a entrada dos termos de busca e filtros de data informados pelo usuário no frontend, os termos e filtros são então enviados para a API REST, que recebe a requisição e realiza a montagem das consultas que serão realizadas no banco de dados SQLite a partir de quais filtros e termos foram informados pelo usuário, a

API então realiza a consulta monta uma resposta com a listagem dos documentos obtidos e suas páginas com conteúdo correspondente para pré visualização. Por fim o frontend recebe a resposta e exibe a listagem de documentos obtidos e uma prévia das páginas que contém o conteúdo textual pesquisado.

3.6 Diagrama de Implantação do Sistema

Durante o desenvolvimento, foi utilizado um servidor VPS para a implantação e testes em um ambiente de execução mais próximo do real. A figura 15 representa o diagrama de implantação do sistema a partir desta implantação feita em desenvolvimento.

Um único servidor foi utilizado para a execução de todos os recursos desenvolvidos, no caso da API REST e do fronted web o gerenciamento é feito usando a ferramenta Docker Compose, usada para a definição da estrutura onde dois conteineres executam as aplicações separadamente, sendo um deles um container rodando o servidor web Nginx que opera para acesso ao frontend e também como proxy reverso para a API.

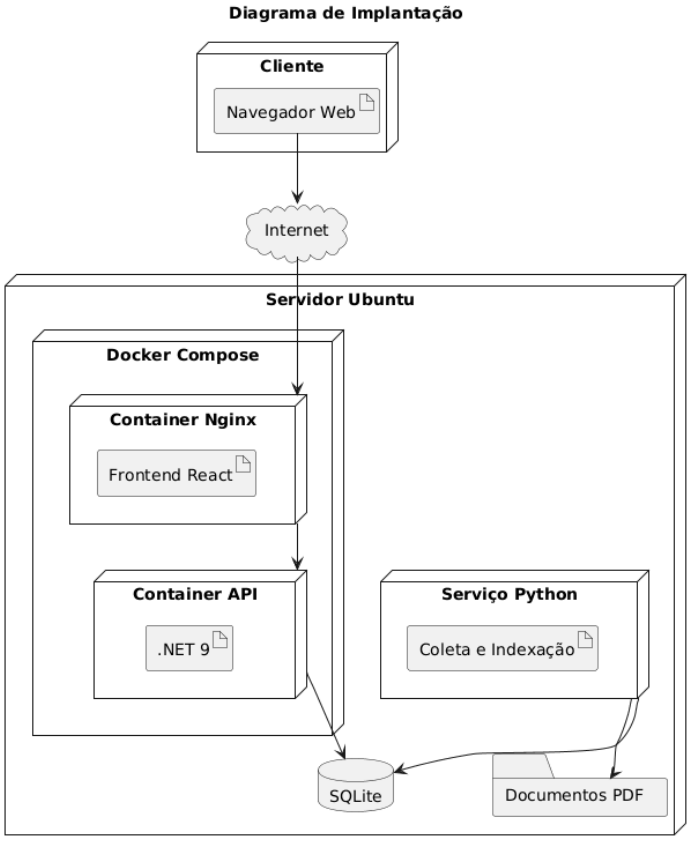


Figura 15 – Diagrama de Implantação do Sistema

Já o serviço python de coleta e indexação opera diretamente no sistema operacional do servidor. A API tem acesso ao banco de dados que é compartilhado entre o sistema operacional do servidor e o container através de um volume compartilhado que é definido

no arquivo YML de estrutura operado pelo Docker Compose, isto permite que o scraper e a API possam acessar a base de dados para realizar as operações de indexação e busca.

No caso do servidor de desenvolvimento, o acesso através da internet aos serviços hospedados foi possível graças ao uso da ferramenta Cloudflare Tunnel, que possibilita a criação de um "tunnel"protegido entre o servidor e a rede da Cloudflare, facilitando a exposição do sistema na web.

3.7 Considerações Sobre o Capítulo

Este capítulo apresentou a análise e o projeto do sistema de recuperação de diários, contendo a modelagem de processos, estrutura de dados, componentes de software e a arquitetura de implantação utilizada. Os diagramas apresentados demonstram a interação entre os componentes do sistema e principais fluxos de execução, passando pela coleta de dados e indexação, até a realização de buscas e apresentação de resultados ao usuário.

A modelagem definiu a separação dos metadados dos documentos obtidos e o conteúdo textual extraído, utilizando o SQLite como ferramenta central utilizando do seu mecanismo de busca textual para a recuperação dos documentos.

Por fim, o diagrama de implantação apresenta a organização dos componentes em ambiente de execução, demonstrando a base de estrutura do sistema e como seus componentes são integrados. O capítulo seguinte aborda aspectos da implementação e funcionamento da solução desenvolvida.

Capítulo 4

Desenvolvimento do Sistema

4.1 Processo de Desenvolvimento

O desenvolvimento se iniciou a partir do módulo de scraping feito em python, foram feitos scripts iniciais que serviram de prova de conceito para a execução das tarefas de scraping do site da prefeitura de Uberlândia. Após a criação dos primeiros scripts e a obtenção dos documentos se mostrar possível, foi realizada a estruturação da base de dados SQLite que serve como base de busca textual.

Desta forma, com os documentos obtidos através das primeiras versões dos scripts python de scraping e a versão inicial do banco de dados, foram criados os primeiros scripts que fazem a extração do conteúdo textual dos documentos e armazena no banco de dados. O que já permitiu a realização de consultas textuais a partir do módulo de full text search do SQLite.

A parte final do desenvolvimento do módulo de scraping se resumiu em refinar e organizar os scripts que até aqui atuavam separadamente de forma a trabalharem em conjunto em um fluxo que permitisse uma execução única e mais automatizada.

Com isto, foi construída uma base para que os outros dois módulos pudessem operar, com a estrutura de banco de dados definida e o módulo de scraping e indexação já funcionando para obter um conjunto inicial de documentos a partir do site da prefeitura de Uberlândia foi possível iniciar a construção da API que fica responsável por consumir a base de dados indexada.

O desenvolvimento da API, assim como da aplicação web de frontend foi executado em menor tempo devido a maior familiaridade com as tecnologias escolhidas, além de que com a estruturação feita durante o desenvolvimento do módulo de indexação foi possível agilizar o desenvolvimento de ambas as ferramentas. Após a conclusão das versões iniciais já funcionais foi realizado algumas etapas de refinamento de código em todos os módulos para correções de bugs e melhorias nas estruturas de projeto e funcionamento geral do sistema.

4.2 Arquitetura do Sistema

O sistema foi desenvolvido utilizando uma arquitetura cliente-servidor de três camadas, composta pela camada de apresentação, camada de serviços e camada de dados. Essa separação permite a organização de responsabilidades entre os componentes do sistema, que possui camadas bem definidas que interoperam.

A camada de apresentação é composta por uma aplicação web desenvolvida usando React e Vite, ela é responsável pela interação do usuário através de uma interface gráfica e seu acesso é feito através de um navegador web.

A camada de serviços é constituída por uma API REST desenvolvida em .NET 9, responsável pelo processamento de requisições vindas da aplicação web, aplicação de regras de negócio e acesso aos dados armazenados. A API disponibiliza endpoints para pesquisa textual, recuperação de diários por id e obtenção de documentos mais recentes.

Já a camada de dados utiliza o banco SQLite que é responsável por armazenar os dados extraídos dos documentos dos Diários Oficiais Municipais, o sistema foi construído de forma que cada cidade indexada possui uma instância de banco sqlite separada, quando uma requisição é feita para buscas de documentos a API recebe por parâmetro qual cidade deve ser acessada, o mesmo ocorre durante o processo de indexação. No caso deste trabalho apenas a cidade de Uberlândia foi utilizada no desenvolvimento, por isso ela é a única presente como possibilidade de seleção no módulo de frontend no estado atual do sistema.

O processo de coleta e indexação é realizado por um web scraper desenvolvido em Python. O módulo de scraping também se limita ao script para o site da prefeitura de Uberlândia em seu estagio atual, mas seu desenvolvimento foi pensado de forma modular para possibilitar a adição de scripts de scraping para os sites de outras prefeituras. O scraper é executado de forma automatizada ou manualmente através de linha de comando e pode buscar arquivos na web ou indexar arquivos que já estejam salvos localmente.

Durante o desenvolvimento, a infraestrutura da aplicação foi implantada em um servidor Ubuntu Server. A API .NET e um servidor web Nginx foram executados em contêineres Docker gerenciados pelo Docker Compose, enquanto o web scraper é executado diretamente no sistema operacional do servidor. O banco de dados SQLite também permanece no sistema de arquivos do próprio servidor e é compartilhado entre a API por meio de um diretório montado como volume no contêiner da API.

O servidor Nginx é responsável por disponibilizar os arquivos estáticos gerados na construção da aplicação React e também por atuar como proxy reverso recebendo as requisições destinadas para a API. O acesso externo é feito através do Cloudflare Tunnel, que estabelece uma conexão segura entre o servidor e a infraestrutura da Cloudflare, permitindo que o serviço seja acessado na web e elimina a necessidade de exposição direta de portas na rede.

Como requisitos mínimos de infraestrutura para execução do sistema, usando como

base a infraestrutura usada durante o desenvolvimento, recomenda-se um servidor com sistema operacional Linux, processador de dois núcleos, 2 GB de memória RAM e aproximadamente 5 GB de espaço livre em disco para armazenamento da aplicação e do banco de dados, além do acesso a internet.

4.3 Tecnologias Utilizadas no Desenvolvimento

A escolha das ferramentas para desenvolvimento do sistema foi focada na facilidade que cada ferramenta oferece para atingir o objetivo de operação de cada camada, além da familiaridade do autor com as mesmas.

No caso da camada de scraping e indexação (dados), o desenvolvimento foi feito usando Python 3.14, sendo as três principais bibliotecas utilizadas, a "pypdf", para leitura e extração textual dos documentos, a "Requests", para obtenção das páginas web, e a "BeautifulSoup", para *parsing* dos html obtidos. Além delas foram utilizadas bibliotecas padrão da linguagem, que contém ferramentas para trabalho com diretórios, arquivos e também acesso ao banco de dados SQLite.

Já na camada de backend (serviços), foi utilizado o framework .NET Core 9, usando linguagem C#, com as principais bibliotecas utilizadas além da biblioteca padrão do framework sendo a "Microsoft.Data.Sqlite", para conexão com a base de dados, "Serilog", para geração de logs, "Scalar", para geração de documentação da API e as bibliotecas "Moq"e "xunit", que são utilizadas nos testes unitários.

Na camada de frontend (apresentação), foi utilizado Typescript através do ReactJs em conjunto com a ferramenta de construção vite para criação da interface de usuário web. Foram usadas duas bibliotecas principais, sendo elas a "tanstack/react-query"para gerenciamento de dados e a "zustand"para gerenciamento de estados.

O banco de dados SQLite é a ferramenta central, já que o recurso de FTS (Full Text Search) do próprio SQLite é o que permite a busca textual dos arquivos nele indexados. A utilização da ferramenta é bastante facilitada por não exigir instalação de nenhuma ferramenta/ambiente para criação e/ou gerenciamento da base de dados, este trabalho pode ser feito diretamente a partir do código das aplicações que utilizam o SQLite como persistência de dados. Outra vantagem é que o espaço de armazenamento é bastante reduzido, o banco existe em apenas um arquivo ".db"que consome pouco espaço em disco mesmo após a inserção de dados. Para exemplificação, durante os testes realizados em desenvolvimento, o espaço ocupado pelos arquivos pdf dos diários oficiais obtidos a partir do site da prefeitura de Uberlândia correspondentes ao período de 01/06/2026 até 23/07/2026 foi de 84,9 MB, já o espaço ocupado pelo banco de dados (arquivo "udi.db") com o conteúdo textual dos documentos pdf já indexados ocupa apenas 14,6 MB.

4.3.1 Linguagens de desenvolvimento

Para o desenvolvimento das aplicações que compõem o sistema foram utilizadas três linguagens de programação: Python, C# e TypeScript.

Tabela 5 – Linguagens de desenvolvimento

Linguagem	Versão	Descrição
Python	3.14	Linguagem de script de alto nível. Usada no módulo de scraping para obtenção e indexação dos documentos.
C#	13.0	Linguagem orientada a objetos pertencente ao ecossistema .NET. Usada no módulo de backend para desenvolvimento da API REST que serve ao frontend web.
TypeScript	6.0.2	Linguagem de script que é uma extensão do JavaScript, adicionando tipagem. Usada no módulo de frontend para o desenvolvimento da pagina web de buscas.

Fonte: O autor (2026).

As linguagens foram escolhidas levando em consideração dois fatores principais, sendo eles, o conhecimento prévio do autor no uso da linguagem e a facilidade de implementação dos recursos necessários em cada módulo. O objetivo foi facilitar ao máximo o desenvolvimento e manutenção dos diferentes módulos da aplicação.

A escolha do Python para o módulo de scraping se deu por ser uma linguagem interpretada e de fácil execução, com ampla quantidade de bibliotecas já existentes que facilitam o trabalho com requisições web (para obtenção dos documentos) e manipulação de arquivos (para leitura e indexação na base de dados), além de possuir o módulo de suporte ao banco de dados SQLite dentro da biblioteca padrão da linguagem.

No caso do módulo de backend, a familiaridade do autor com o ambiente .NET foi o principal fator de escolha, o que permitiu maior agilidade no desenvolvimento deste módulo. Por fim, no módulo de fronted, o TypeScript foi escolhido por ser uma ferramenta amplamente utilizada no contexto do desenvolvimento web moderno.

4.4 Padrões de Projetos Utilizados

No desenvolvimento do sistema foram utilizadas abordagens e padrões de projeto com o objetivo de organizar e facilitar a manutenção das aplicações envolvidas nos diferentes módulos, organizando a estrutura de código e separando responsabilidades. Os principais padrões utilizados estão relacionados à API do módulo backend, com separação de camadas, regras de negócio e acesso à base de dados.

O padrão Repository foi utilizado na camada de acesso aos dados, esta abordagem concentra as operações de leitura e manipulação do banco de dados em classes específi-

cas, evitando que as outras camadas da aplicação tenham acesso direto aos mecanismos relacionados a persistência dos dados.

A aplicação também utiliza uma camada de serviços, operada por uma classe de serviço que é responsável por intermediar a comunicação entre os controladores (que são responsáveis pelas requisições) e a camada de repositório. Esta abordagem permite concentrar a aplicação das regras de operação em uma camada específica.

O padrão de Injeção de dependência é utilizado para controlar as dependências entre os componentes da API. Este padrão permite que as classes não precisem criar diretamente suas dependências, este trabalho é feito pelo mecanismo de injeção de dependência do ASP.NET Core.

Outro padrão utilizado foi o de uso de modelos de transferência de dados, que é a criação de modelos específicos para representar os dados recebidos e enviados pela API, separando as estruturas de requisição e resposta, além da entidade de dados dos diários. Permitindo o controle de quais informações são recebidas e disponibilizadas nos endpoints da aplicação.

Como já citado, a implementação faz uso do padrão de organização em camadas, separando as responsabilidades entre controllers, services e repositories. Os controladores são responsáveis pelo recebimento das requisições e retorno de respostas, a camada de serviço cuida das operações e regras do sistema e a camada de repositório do acesso aos dados. Esta organização permite reduzir o acoplamento entre partes da aplicação e facilita o desenvolvimento e manutenção, permitindo que uma camada seja modificada ou evolua de maneira independente.

4.4.1 Bibliotecas

Para as bibliotecas utilizadas nas aplicações desenvolvidas, além das bibliotecas padrão de cada ferramenta utilizada para o desenvolvimento, algumas bibliotecas externas foram utilizadas para obtenção de melhores resultados. Em todas as ferramentas desenvolvidas neste trabalho foram utilizadas bibliotecas auxiliares, as mais notáveis estão listadas na tabela X:

Tabela 6 – Bibliotecas utilizadas

Ferramenta	Módulo	Função
pypdf	Scraping	Manipulação de arquivos PDF
BeautifulSoup	Scraping	Extração de dados de arquivos HTML
Microsoft.Data.Sqlite	API Backend	Interação com base de dados SQLite no sistema .NET
Scalar	API Backend	Criação de documentação interativa
react-query	Frontend	Gerenciamento de estado assíncrono e busca de dados na API
zustand	Frontend	Gerenciamento de estado global de dados

Fonte: O autor (2026).

4.4.2 Ferramentas Auxiliares

As principais ferramentas auxiliares utilizadas foram o Visual Studio, que é uma IDE (Ambiente de Desenvolvimento Integrado) voltada para o desenvolvimento de aplicações do ecossistema .NET, usada para o desenvolvimento da API do módulo de backend, e o Visual Studio Code, que é um editor de texto voltado para o desenvolvimento de software com capacidade para oferecer suporte a diferentes linguagens e ambientes de desenvolvimento, nele foram desenvolvidos os módulos de scraping e frontend.

Além dos editores, é possível listar também a ferramenta git, usada para o versionamento de código dos projetos.

4.4.3 Sistemas e componentes externos utilizados

O uso de sistemas e componentes complementares ficou mais direcionado na implantação do sistema em um servidor próprio que foi utilizado durante o desenvolvimento. Dentre estes, os mais relevantes a serem citados são: Docker, Nginx, Ubuntu Server e Cloudflare Tunnels.

Tabela 7 – Sistemas e componentes externos

Componente	Descrição
Docker	Plataforma utilizada para executar a API .NET e o servidor web Nginx em contêineres isolados
Nginx	Servidor web responsável por disponibilizar os arquivos estáticos do frontend React e atuar como proxy reverso, encaminhando as requisições destinadas à API para o contêiner correspondente.
Ubuntu Server	Sistema operacional do servidor utilizado para hospedar a aplicação, executar os contêineres Docker e disponibilizar o ambiente necessário para a execução do processo de coleta e indexação dos documentos.
Cloudflare Tunnels	Serviço utilizado para estabelecer uma conexão segura entre o servidor de origem e a infraestrutura da Cloudflare, permitindo disponibilizar a aplicação externamente por meio de um domínio público sem exigir a exposição direta das portas do servidor à Internet.

Fonte: O autor (2026).

4.5 Instalação e Configurações

Nesta seção é descrita o processo de instalação e configuração das aplicações envolvidas no sistema de indexação e busca de diários, o sistema funciona como uma aplicação web

e sua implementação é composta em quatro partes:

1. Scraper em Python: responsável pela coleta e extração do conteúdo dos arquivos.
2. API REST em .NET: responsável por disponibilizar os dados indexados e executar as consultas realizadas pelo frontend web.
3. Frontend em React + TypeScript: responsável pela interface de busca e apresentação de resultados.
4. Configuração do ambiente de execução.

A persistência dos dados é feita através do SQLite, com utilização do mecanismo SQLite FTS (Full-Text Search) para as consultas textuais. Durante o desenvolvimento, a aplicação foi hospedada em um servidor Ubuntu utilizando Docker para executar os componentes da API REST e o frontend React, já o scraper é executado diretamente no sistema operacional do servidor, onde um script de execução foi configurado para que o processo de busca e indexação aconteça de forma automatizada durante o período de testes da aplicação. Os componentes do sistema de busca possuem arquivos de configuração separados, durante a implantação se faz necessário ajustar estes arquivos de acordo com o ambiente em que o sistema está sendo executado, a principal razão disto é que os diferentes módulos do sistema compartilham o diretório de dados onde os arquivos SQLite são armazenados.

As descrições de instalação e configuração deste trabalho são feitas a partir do processo de implantação que foi realizado durante o desenvolvimento e testes do sistema de buscas em um servidor próprio, é possível executar as aplicações individualmente diretamente no sistema operacional sem a necessidade do processo aqui descrito utilizando Docker e Nginx, porém, a descrição apresentada nas seções seguintes buscam se aproximar de um ambiente real de execução do sistema, utilizando Docker e Nginx para a execução das aplicações e distribuição web.

4.5.1 Pré-requisitos do Sistema

A execução do sistema requer uma infraestrutura capaz de hospedar os componentes que fazem parte do sistema de busca. O ambiente de implantação deste trabalho utiliza um servidor com sistema operacional Ubuntu Server, Docker e Docker compose para execução dos componentes web da aplicação. Além destes, é possivel também listar algumas outras ferramentas utilizadas:

Fonte: O autor (2026).

O processo de coleta dos documentos requer a instalação do Python e um ambiente virtual destinado ao isolamento das dependências utilizadas pelo scraper diretamente no servidor, o processo de configuração e os scripts de automação através do systemd são

Tabela 8 – Componentes usados na execução

Componente	Ferramenta	Finalidade
Sistema Operacional	Ubuntu Server	Hospedagem do sistema
Contêinerização	Docker + Docker Compose	Execução da API e Nginx
Servidor web	Nginx	Frontend e proxy reverso
Coleta	Python 3	Scraping dos Diários
Automação	systemd	Execução automática do scraper
Acesso externo	Cloudflare Tunnel	Publicação do sistema na Internet

descritos mais adiante. Além disso, o acesso a internet é necessário para que o serviço de coleta possa buscar os documentos nos repositórios do site da prefeitura.

O servidor precisa ter, no mínimo, para a implantação do sistema, a capacidade de execução do Docker e Docker Compose, Python 3 e ambiente virtual, espaço em disco suficiente para armazenamento dos arquivos e banco de dados dos Diários Oficiais, permissão de leitura e escrita no diretório utilizado pelo banco de dados e acesso a internet para o acesso aos portais municipais e disponibilização do sistema externamente.

Já para o ambiente de desenvolvimento/compilação dos componentes do sistema de busca são necessários algumas ferramentas, sendo elas:

- ❏ Frontend
  - Node.js
  - npm
  - dependências do projeto
  - Vite
- ❏ Backend
  - .NET SDK 9
  - dependências NuGet do projeto
- ❏ Scraper
  - Python 3
  - pip
  - ambiente virtual Python
  - dependências listadas no requirements.txt

Para o usuário final, não são necesários componentes adicionais instalados. O acesso ao sistema pode ser feito através de um navegador web pela internet, uma vez que os componentes de processamento são executados no servidor.

4.5.2 Processo de Instalação/Execução

O processo de instalação/execução é dividido em partes, seguindo a separação dos módulos do sistema, cada aplicação é compilada/construída separadamente, depois os fragmentos que resultam do processo de compilação são enviados ao servidor onde as demais configurações necessárias são realizadas.

4.5.2.1 Frontend

Para o frontend, durante o desenvolvimento, a aplicação utiliza o servidor de desenvolvimento do Vite. Para a implantação, no entanto, não é necessário que o servidor possua Node.js ou Vite sendo executado. O processo de publicação é realizado da seguinte maneira:

1. O processo de construção é feito na máquina de desenvolvimento com o comando:

```
npm run build
```

2. O comando de construção gera o diretório com os arquivos estáticos da aplicação:

```
js_diarios_ui/dist/
```

3. O diretório contendo os arquivos da aplicação é copiado para o servidor em um diretório que é posteriormente acessado pelo Nginx para disponibilização da aplicação na web.

Portanto, o ambiente no servidor não executa código React diretamente. Os arquivos Typescript são transformados em arquivos estáticos que são disponibilizados pelo servidor web. O contêiner do Nginx ("diarios-ui") recebe o diretório com os arquivos estáticos montado em um volume somente leitura no contêiner, dessa forma o Nginx disponibiliza diretamente os arquivos gerados pelo comando de construção do frontend, essa estrutura permite que ao gerar novas versões da aplicação, os novos arquivos resultantes do processo de construção apenas substituam os arquivos antigos no diretório do servidor, assim o Nginx passa a servir os novos arquivos na web.

4.5.2.2 Backend

Para o backend, o desenvolvimento fica contido à IDE do Visual Studio, onde a aplicação é desenvolvida e testada localmente em um ambiente integrado. Partindo para o processo de implantação, sua publicação é semelhante ao processo do frontend:

1. O processo de construção é feito na máquina de desenvolvimento com o comando:

```
dotnet publish -c Release
```

2. O comando de construção gera o diretório com os arquivos estáticos da aplicação.

```
dotnet_diarios_api/Diarios.Api/bin/Release/net9.0/publish/
```

3. O diretório contendo os arquivos da aplicação é copiado para o servidor em um diretório que é montado como um volume no contêiner da API.

4. O arquivo de configuração da aplicação é ajustado conforme a estrutura de diretórios da implantação.

O diretório gerado pela construção contém o assembly principal da aplicação, este diretório é compartilhado em um volume com o contêiner da API onde a aplicação é executada, este contêiner não fica exposto na web, o contêiner do Nginx redireciona chamadas feitas para a API pela web para a aplicação. No caso da API, uma configuração adicional é necessária após a implantação pois a aplicação precisa acessar o diretório onde está armazenada a base de dados SQLite, esta configuração é descrita nas seções seguintes.

4.5.2.3 Scraper

Para o módulo do scraper o processo é um pouco diferente dos dois módulos anteriores, o scraper utiliza um ambiente virtual próprio para evitar que suas dependências sejam instaladas globalmente no sistema operacional, isto vale tanto para o ambiente de desenvolvimento como para o ambiente de implantação. Por se tratar de uma linguagem interpretada, não é necessário que seja feita a compilação dos arquivos de código da aplicação, a execução pode ser feita diretamente ao passar o arquivo de script principal do scraper para o interpretador da linguagem Python. O repositório do código do scraper possui um arquivo "requirements.txt"que permite a ferramenta "pip"do Python obter as dependências necessárias do projeto dentro do ambiente virtual criado.

Após a criação do ambiente virtual e obtenção das dependências do projeto, o scraper deve ser executado utilizando do interpretador Python presente no diretório do ambiente virtual, permitindo que o projeto tenha acesso as dependências previamente definidas. Além disso, uma configuração adicional é necessária no arquivo de configuração presente no diretório do projeto do scraper, para certificação de que o scraper está acessando o mesmo diretório de base dadaos que a API, já que o banco de dados é compartilhado entre os dois módulos do sistema de buscas. Esta configuração é demonstrada nas seções seguintes.

4.5.2.4 Servidor

Antes deste processo, a preparação no servidor começa pela criação dos contêiners que vão executar os componentes web da aplicação. A arquitetura possui dois contêiners principais:

1. diarios-api
2. diarios-ui

O contêiner da API utiliza a imagem oficial disponibilizada pela Microsoft para a execução de aplicações .NET, já o contêiner do frontend utiliza a imagem oficial de build do Nginx, isso permite que a execução seja feita sem a necessidade de construção de uma imagem específica da aplicação, o arquivo de configuração contendo a definição desta estrutura é mostrado no bloco de código 4.3 mais a diante.

A figura 16 demonstra a estrutura de diretórios que foi criada para o sistema no servidor durante o desenvolvimento do trabalho. Os diretórios "backend"e "frontend"que contém os arquivos gerados da compilação dos aplicações serão montados em volumes nos contêineres, assim como o diretório "data"que também é montado no contêiner da API permitindo que a base de dados SQLite possa ser acessada pela aplicação.

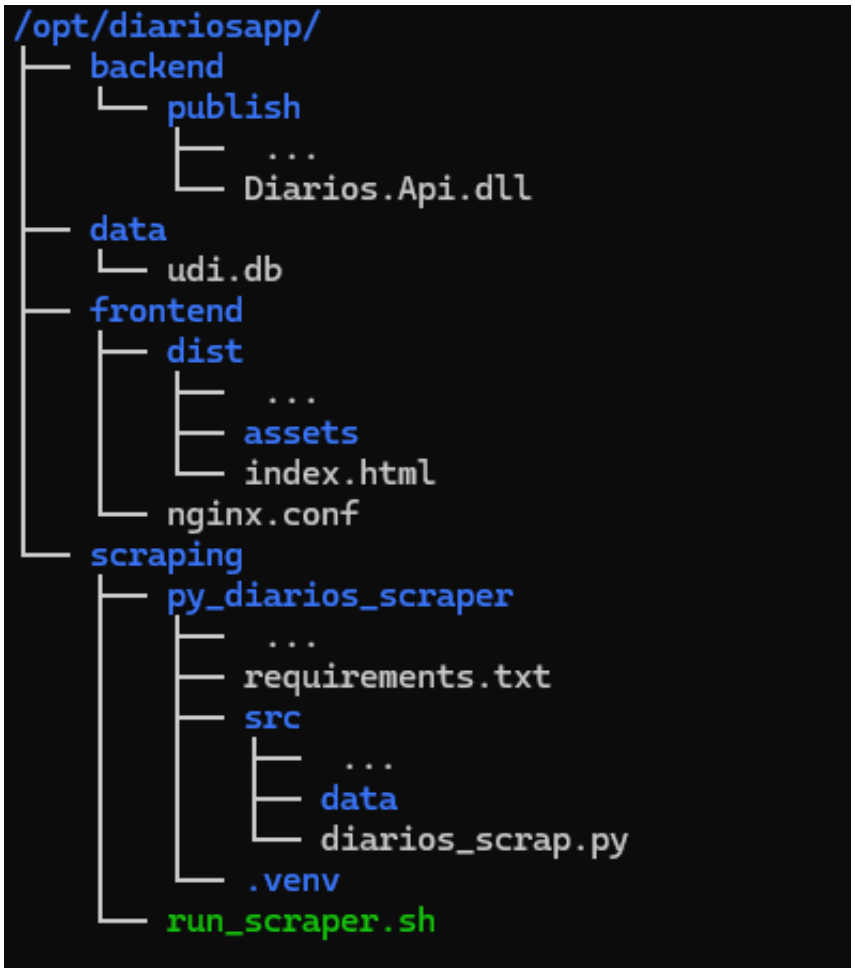


Figura 16 – Estrutura de Diretórios no servidor

Fonte: O autor (2026).

Para a execução automatizada do scraper, é possível utilizar diferentes serviços de agendamento de execução, no caso deste trabalho foi utilizada a ferramenta de serviço

do systemd que gerencia a execução de processos em segundo plano no Linux. Para isso, foram criados dois arquivos: "diarios-scraper.service" e "diarios-scraper.timer" que estão presentes no seguinte diretório:

```
/etc/systemd/system/
```

Para ativação dos scripts de execução são utilizados os comandos:

```
sudo systemctl daemon-reload
sudo systemctl enable diarios-scraper.timer
sudo systemctl start diarios-scraper.timer
```

Ambos os arquivos de configuração estão especificados mais adiante nos blocos de código 4.5 e 4.6.

4.5.3 Variáveis de Ambientes Necessárias

Toda a configuração dos módulos do sistema é feita através de arquivos de configuração listados, não é necessário que sejam definidas variáveis de ambiente diretamente no sistema que está executando as aplicações (((NO ARQUIVO DE CONFIGURAÇÃO DO CONTAINER DA API EXISTE - REVISAR))). Abaixo está listada as principais variáveis de configuração e quais módulos e arquivos elas pertencem:

Tabela 9 – Variáveis de ambiente

Variável	Módulo	Arquivo	Descrição
[parser].MenorAno	Scraper	config.ini	Define o ano limite que o scraper pode buscar
[parser.cidades]	Scraper	config.ini	Lista de cidades configuradas para indexação de documentos
[logs].Path	Scraper	config.ini	Caminho onde o script de logs do scraper deve acessar para armazenar os registros
[data].DownloadsDir	Scraper	config.ini	Caminho onde o scraper deve acessar para armazenar os arquivos pdf dos diarios
[data].DataBaseDir	Scraper	config.ini	Caminho onde o scraper deve armazenar a base de dados indexada
ConnectionStrings	API	appsettings.env.json	Caminho onde a API deve encontrar o arquivo de base de dados de determinada cidade

Fonte: O autor (2026).

4.5.4 Arquivos de Configuração

A maior parte dos arquivos de configuração está relacionada a implantação do sistema no servidor, os blocos de código 4.1 à 4.7 apresentam o conteúdo destes arquivos com trechos de exemplo de configuração.

Código 4.1 – config.ini

```
[parser]
MenorAno = 2015

[parser.cidades]
Uberlandia = udi

[logs]
Path = /opt/diariosapp/scraping/py_diarios_scraper/src/logs/

[data]
DownloadsDir = /opt/diariosapp/scraping/py_diarios_scraper/src/
   downloads/
DataBaseDir = /opt/diariosapp/data/
```

Fonte: O autor (2026).

Código 4.2 – appsettings.env.json

```
{
  "ConnectionStrings": {
    "udi": "Data Source=/data/udi.db",
    "monte": "Data Source=/data/monte.db"
  },
}
```

Fonte: O autor (2026).

Os códigos 4.1 e 4.2 apresentam os arquivos de configuração do scraper e da API backend respectivamente. No caso do código do arquivo de configuração do scraper existem diferentes variáveis que podem ser utilizadas (descritas na tabela 9), as duas mais relevantes são as variáveis "DownloadsDir"e "DataBaseDir"do campo "[data]", que definem os caminhos dos diretórios de download dos arquivos pdf dos diários oficiais obtidos e da base de dados SQLite onde a indexação é feita.

Já o arquivo de configuração referente à API contém as strings de conexão que apontam para as bases de dados de cada cidade configurada. Aqui o caminho é correspondente ao do diretório que é montado no volume do contêiner da API no Docker, já que durante a execução a API só "enxerga"os diretórios montados no contêiner onde ela é executada.

Código 4.3 – docker-compose.yml

```
1  services:
2    api:
3      image: mcr.microsoft.com/dotnet/aspnet:9.0
4      container_name: diarios-api
5      working_dir: /app
6      command: dotnet Diarios.Api.dll
7      environment:
8        ASPNETCORE_ENVIRONMENT: Development
9        ASPNETCORE_URLS: http://+:8080
10     expose:
11       - "8080"
12     volumes:
13       - ./backend/publish:/app:ro
14       - ./data:/data
15     restart: unless-stopped
16
17   frontend:
18     image: nginx:alpine
19     container_name: diarios-ui
20     volumes:
21       - ./frontend/dist:/usr/share/nginx/html:ro
22       - ./frontend/nginx.conf:/etc/nginx/conf.d/default.conf:ro
23     ports:
24       - "80:80"
25     restart: unless-stopped
26     depends_on:
27       - api
```

Fonte: O autor (2026).

O código 4.3 apresenta o arquivo de configuração do Docker, nele é definida a estrutura dos contêineres onde a API e o módulo de frontend são executados. A estrutura separa os dois serviços e define os volumes que são montados no sistema de arquivos de cada contêiner, no caso da API são os volumes correspondentes ao diretório onde os arquivos da aplicação foram copiados no servidor (linha 13) e o diretório onde as bases de dados SQLite estão (linha 14). Já no módulo de frontend os volumes são correspondentes ao diretório com os arquivos estáticos que foram copiados para o servidor (linha 21) e o caminho do arquivo de configuração do Nginx (linha 22).

Código 4.4 – nginx.conf

```
1  server {
2      listen 80;
```

```
3     server_name _;
4
5     root /usr/share/nginx/html;
6     index index.html;
7
8     location / {
9         try_files $uri $uri/ /index.html;
10    }
11
12    location /api {
13        proxy_pass http://api:8080;
14
15        proxy_http_version 1.1;
16
17        proxy_set_header Host $host;
18        proxy_set_header X-Forwarded-For
    $proxy_add_x_forwarded_for;
19        proxy_set_header X-Forwarded-Proto $scheme;
20    }
21 }
```

Fonte: O autor (2026).

O arquivo de configuração do Nginx (código 4.4) define a estrutura do serviço que funciona como servidor http para o frontend e proxy reverso para a API, permitindo que as chamadas externas sejam atendidas.

Os arquivos de configuração apresentados até aqui estão direcionados ao funcionamento dos módulos da aplicação e sua disponibilização. Para a execução automatizada do scraper foi feito o uso do systemd para o agendamento de execução do script, os blocos de código a seguir apresentam os arquivos utilizados nesta funcionalidade.

Código 4.5 – diarios-scraper.service

```
1 [Unit]
2 Description=Coleta de diarios
3
4 [Service]
5 Type=oneshot
6 User=lmf
7 WorkingDirectory=/opt/diariosapp/scraping/py_diarios_scraper
8 ExecStart=/opt/diariosapp/scraping/run_scraper.sh
```

Fonte: O autor (2026).

Código 4.6 – diarios-scraper.timer

```
1 [Unit]
```

```
Description=Agendamento da coleta de diarios

[Timer]
OnCalendar=*-*-* 23:55:00
Persistent=true

[Install]
WantedBy=timers.target
```

Fonte: O autor (2026).

Os blocos de código 4.5 e 4.6 apresentam os arquivos de configuração que definem o agendamento da execução do script que gera o comando de busca por diários, no exemplo apresentado esta execução é realizada diariamente após às 23:55.

Código 4.7 – run-scraper.sh

```
#!/bin/bash

ANO=$(date +%Y)
MES=$(date +%-m)

/opt/diariosapp/scraping/py_diarios_scraper/.venv/bin/python \
/opt/diariosapp/scraping/py_diarios_scraper/src/diarios_scrap.py
   -i -dl -a "$ANO" -m "$MES" -c udi
```

Fonte: O autor (2026).

O código 4.7 contém a definição do script que executa a chamada do scraper. O script utiliza o interpretador do python que está dentro do ambiente virtual criado para a execução do módulo, além de obter a data atual que é passada como parâmetro na chamada.

4.6 Utilização do Sistema

...

4.6.1 Processo de Execução do Sistema

...

4.6.2 Exemplos de Utilização das Principais Funcionalidades do Sistema

...

4.6.2.1 Requisito Funcional 001

...

4.6.2.2 Requisito Funcional 002

... ...

4.7 Testes do Sistema

...

4.7.1 Plano de Testes

...

4.7.2 Execução do Plano de Testes <Opcional>

...

4.8 Repositório de Código

As ferramentas desenvolvidas no trabalho foram disponibilizadas em repositórios públicos separados no github do autor, os links de acesso podem ser encontrados na tabela 10.

4.8.1 Link para o Repositório

Tabela 10 – Links de repositórios

Ferramenta	Acesso
Scraper/Indexador	https://github.com/leowbk77/py_diarios_scraper
API REST	https://github.com/leowbk77/dotnet_diarios_api
Frontend web	https://github.com/leowbk77/js_diarios_ui

Fonte: O autor (2026).

4.8.2 Estrutura de Diretórios

4.8.3 Guia de Contribuições (se aplicável)

...

4.9 Resultados Alcançados com o Sistema

...

4.10 Considerações Sobre o Capítulo

...

Capítulo 5

Conclusões

5.1 Principais Contribuições

5.2 Trabalhos Futuros

Referências

BAEZA-YATES, R.; RIBEIRO-NETO, B. **Recuperação de Informação: Conceitos e Tecnologia das Máquinas de Busca**. [S.l.]: Bookman, 2013. v. 2. Citado 2 vezes nas páginas 9 e 10.

BRASIL. Lei nº 12.527, de 18 de novembro de 2011. regula o acesso a informações previsto no inciso xxxiii do art. 5º , no inciso ii do § 3º do art. 37 e no § 2º do art. 216 da constituição federal; altera a lei nº 8.112, de 11 de dezembro de 1990; revoga a lei nº 11.111, de 5 de maio de 2005, e dispositivos da lei nº 8.159, de 8 de janeiro de 1991; e dá outras providências. **Diário Oficial [da] República Federativa do Brasil**, Brasília, DF, 2011. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2011-2014/2011/lei/l12527.htm>. Citado na página 10.

ISO-8601. **Date and time - Representations for information interchange**. 1. ed. Geneva, Switzerland, 2019. ISO 8601-1:2019. Citado na página 28.

MINAS, P. de. **Portal de acesso aos Diários Oficiais Municipais de Patos de Minas**. 2025. Disponível em: <https://www.patosdeminas.mg.gov.br/portal/diario-oficial>. Citado na página 11.